



DelayShield

DELIVERY RISK INTELLIGENCE

Explainable machine learning for delivery-delay risk prediction and supplier prioritization, built end-to-end on the Brazilian E-Commerce Public Dataset by Olist.

PREDICT



EXPLAIN



PRIORITIZE



RECOMMEND



SIMULATE



DECIDE

Project Report — Minor Project, SASTRA
University

Dataset: Brazilian E-Commerce Public Dataset by
Olist (99,441 orders)

Model trained: 2026-08-20

Table of Contents

1. Executive Summary	3
2. Problem Statement & Objective	3
3. Dataset	4
4. System Architecture & Pipeline	4
5. Target Definition & Leakage Control	5
6. Feature Engineering	6
7. Multi-Seller Order Handling	7
8. Model Development	7
9. Chronological Train/Validation/Test Split	7
10. Class Imbalance Handling	8
11. Model Comparison & Selected Model	8
12. Evaluation Metrics (Measured Results)	9
13. Risk Classification	10
14. Explainable AI (SHAP)	10
15. Supplier Prioritization Engine	11
16. Action Recommendation Engine	11
17. Business Cost Model & Policy Simulation	12
18. Streamlit Dashboard	13
19. Project Structure	14
20. Limitations	14
21. Reviewer Q&A	15
22. Conclusion & Future Work	16

1. Executive Summary

DelayShield is a decision-support system that predicts, at order confirmation, the probability that a delivery will miss its promised delivery date. Rather than stopping at a classifier, it forms a complete operational loop: predict the risk, explain the prediction with SHAP, rank suppliers on a weighted scorecard, recommend a concrete operational action, and simulate a proactive policy against the current reactive status quo under transparent, editable cost assumptions.

The system is built entirely on the **Brazilian E-Commerce Public Dataset by Olist** (99,441 orders, 2016–2018) — the dataset explicitly named in the approved project proposal. Of these, 96,470 delivered orders with complete timestamps form the modelling population, of which 8.11% were delivered after their promised date.

ORDERS MODELLED 96,470 delivered, complete timestamps	OVERALL LATE RATE 8.11% 7,826 late orders	SELECTED MODEL Random Forest threshold-tuned, no class weighting	TEST ROC-AUC 0.726 32,150 held-out orders
--	--	--	--

Three model families (Logistic Regression, Random Forest, XGBoost) were trained, each with and without class-weighted imbalance handling — six configurations total, all reported. Evaluation used a strictly **chronological** train/validation/test split to avoid the model learning from future information. The final model is served through a seven-page Streamlit dashboard styled as an enterprise supply-chain risk platform.

Every number in this report is a measured value from an actual training run of this codebase — none are illustrative or rounded up for effect. Where the model's performance is limited (e.g. alert precision), that limitation is stated plainly rather than hidden.

2. Problem Statement & Objective

2.1 Business Problem

Late deliveries damage customer trust, increase support load, and are typically discovered only after the fact — when it is too late to do anything except apologise. A system that flags *at order confirmation* which orders are at risk of running late allows a business to act proactively: buffer the schedule, contact the customer ahead of time, escalate tracking, or route around an unreliable supplier.

2.2 Project Objective

The approved proposal defines DelayShield as a system that must:

- Predict delivery-delay probability at order confirmation, using only information available at that moment;
- Explain each prediction (SHAP / feature importance);
- Classify orders into LOW / MEDIUM / HIGH risk;
- Prioritize and score suppliers on historical reliability and predicted risk;
- Recommend concrete operational actions per risk level;
- Simulate a proactive risk-aware policy against a reactive baseline under transparent cost assumptions;
- Present all of the above through an interactive Streamlit dashboard.

Design identity. DelayShield is deliberately built as a PREDICT → EXPLAIN → PRIORITIZE → RECOMMEND → SIMULATE → DECIDE decision-support pipeline, not a standalone classification notebook.

3. Dataset

3.1 Source

Brazilian E-Commerce Public Dataset by Olist, obtained as the original (non-pre-merged) release of nine relational CSV files covering orders placed on the Olist marketplace between 2016 and 2018.

Table	File	Rows	Columns	Missing cells
orders	olist_orders_dataset.csv	99,441	8	0.62%
order_items	olist_order_items_dataset.csv	112,650	7	0.00%
customers	olist_customers_dataset.csv	99,441	5	0.00%
sellers	olist_sellers_dataset.csv	3,095	4	0.00%
products	olist_products_dataset.csv	32,951	9	0.83%
payments	olist_order_payments_dataset.csv	103,886	5	0.00%
reviews	olist_order_reviews_dataset.csv	99,224	7	21.01%
geolocation	olist_geolocation_dataset.csv	1,000,163	5	0.00%
category_translation	product_category_name_translation.csv	71	2	0.00%

3.2 Modelling population

Only orders with `order_status == 'delivered'` and all three required timestamps (purchase, delivered, estimated delivery) present are used. Orders where the delivery date precedes the purchase date are dropped as data-quality errors. This filtering yields **96,470** usable orders spanning **2016-09-15 to 2018-08-29**.

3.3 Reviewer question: why Olist?

It is the dataset explicitly named in the approved proposal, and it is one of the few large public e-commerce datasets that carries both a promised delivery date and an actual delivery date per order, alongside seller, product, freight, customer-location and review data — everything the proposal's feature requirements call for.

4. System Architecture & Pipeline

The system is organised as nine focused modules under `src/`, executed by a single training script, and served by a Streamlit dashboard:

```
Olist Dataset (9 CSVs)
|
v
data_loader.py      raw table loading, geolocation centroids
|
v
preprocessing.py    clean, filter to delivered orders, build target,
                    aggregate multi-seller items, join customer/seller/
                    geo/review data, compute distance proxy
|
v
feature_engineering.py time-aware lagged features (seller/category/route
                    history, workload, interactions) - LEAKAGE-FREE
|
v
train.py            chronological split, 6 model configs, threshold
                    tuning, risk-band derivation, artefact persistence
|
v
predict.py          single-order / batch scoring engine
explain.py           SHAP local + global explanations
supplier_scoring.py  seller scorecard, configurable weighted score
recommendations.py   rule-based action playbook
simulation.py        reactive vs risk-aware policy comparison
|
v
dashboard/app.py    7-page Streamlit application
```

Every dashboard page reads only from persisted artefacts (`models/delay_model.pkl` , `model_metadata.json` , `scored_orders.csv` , `supplier_metrics.csv`) — the UI never re-trains anything, so the app is fast to open and its numbers exactly match the training run.

5. Target Definition & Leakage Control

5.1 Target

```
delivery_delay = 1 if order_delivered_customer_date > order_estimated_delivery_date
delivery_delay = 0 otherwise
```

This is the actual SLA breach a customer experiences — it needs no arbitrary threshold, and both timestamps exist for every delivered order.

5.2 Two leakage rules

Rule 1 — no post-outcome columns as features. The actual delivery timestamp, carrier hand-over date, approval timestamp, review score and order status are never model inputs. They are used only to construct the target, the lagged history features, and the retrospective supplier scorecard.

Rule 2 — no look-ahead inside history features. A seller's historical delay rate for an order placed on 2018-03-01 is computed using only that seller's orders whose outcome (`order_delivered_customer_date`) was already known — i.e. strictly before 2018-03-01. An order placed two days earlier but not yet delivered contributes nothing. This is implemented in `_lagged_outcome_stats()` by sorting each group by the moment its outcome became known and using a binary search (`searchsorted`) against the current purchase timestamp — a stricter guarantee than a simple expanding mean over prior rows.

5.3 Cold-start fallback

New sellers or categories with no delivered history (5.5% of orders) fall back to the *running platform-wide delay rate at that moment in time* — never a full-dataset average, which would itself leak future information.

5.4 Workload features are legitimate

`seller_orders_last_30_days` , `seller_orders_last_90_days` , `seller_workload_surge` and `platform_orders_last_30_days` count *purchase events*, not outcomes. Purchase events are known immediately, so these are honestly available at confirmation time and are not subject to Rule 2's lag.

6. Feature Engineering

41 features total: 38 numeric, 3 categorical (one-hot encoded, rare levels grouped under a frequency floor).

Group	Features
Order timing	<code>order_purchase_month/week/day/weekday/hour/quarter</code> , <code>is_weekend</code>
Promise	<code>estimated_lead_time_days</code> , <code>shipping_limit_days</code> , <code>lead_time_slack_days</code> , <code>lead_time_ratio</code>
Commercial	<code>total_order_value</code> , <code>total_freight_value</code> , <code>freight_ratio</code> , <code>item_count</code> , <code>unique_product_count</code> , <code>seller_count</code> , <code>max_item_price</code>
Product	<code>product_weight_g</code> , <code>product_volume_cm3</code>
Location	<code>distance_km</code> (great-circle, zip-prefix centroids), <code>same_state</code> , <code>customer_zip_code_prefix</code> , <code>distance_per_promised_day</code>
Seller history (lagged)	<code>seller_hist_orders</code> , <code>seller_hist_delay_rate</code> , <code>seller_hist_on_time_rate</code> , <code>seller_hist_avg_delivery_days</code> , <code>seller_hist_avg_delay_days</code> , <code>seller_hist_avg_freight</code>
Workload	<code>seller_orders_last_30/90_days</code> , <code>seller_workload_surge</code> , <code>platform_orders_last_30_days</code>
Context history (lagged)	<code>category_hist_delay_rate</code> , <code>customer_state_hist_delay_rate</code> , <code>route_hist_delay_rate</code> , <code>platform_hist_delay_rate</code>
Categorical	<code>product_category</code> , <code>customer_state</code> , <code>seller_state</code>

Strongest engineered signal. `lead_time_slack_days` (promised lead time minus the seller's own historical average delivery time) and `distance_per_promised_day` rank among the highest mean-|SHAP| features globally — the model largely learns to detect *when the promised date is optimistic relative to this seller's own track record on this route*, rather than relying on raw distance or raw lead time alone.

7. Multi-Seller Order Handling

1,275 of 96,470 orders (1.3%) contain items from more than one seller. The project does not assume one order maps to one seller:

- the order-level row retains `seller_count` as a model feature;
- the order is attributed to a **primary seller** — whichever seller holds the largest share of the order's total value (ties broken by the first item id) — whose lagged history drives the seller-level features;
- supplier prioritization runs on a **separate seller-level scorecard** (`supplier_scoring.py`), so a multi-seller order still contributes to every relevant seller's own aggregate there, independent of who was "primary" for the delay model.

8. Model Development

Three model families were trained, matching the proposal's comparison requirement:

Model	Configuration
Logistic Regression	Baseline. L2-regularised ($C=0.5$), standardised numeric features, <code>max_iter=2000</code> .
Random Forest	400 trees, <code>min_samples_leaf=20</code> , <code>max_features='sqrt'</code> .
XGBoost	350 trees, <code>max_depth=4</code> , <code>learning_rate=0.05</code> , <code>min_child_weight=20</code> , <code>reg_lambda=3.0</code> , histogram tree method.

Each family was trained **twice** — once with balanced class weighting and once with threshold tuning only — giving six configurations in total, all reported in Section 11.

9. Chronological Train/Validation/Test Split

A random split would let the model see future orders, and the same seller's future performance, which is impossible at real prediction time and would inflate every reported metric. DelayShield uses a strict chronological split instead:

Split	Rows	Date range	Late rate
Train	43,693	2016-09-15 → 2017-12-31	6.60%
Validation	20,627	2018-01-01 → 2018-03-31	14.58%
Test	32,150	2018-04-01 → 2018-08-29	6.02%

Regime shift, stated plainly. The validation window's late rate (14.58%) is more than double the test window's (6.02%) — a real disruption in Brazilian carrier performance during early 2018. Thresholds tuned on validation therefore do not transfer perfectly to test; this is discussed quantitatively in Section 17.

Validation is used for model selection and threshold tuning only; the test set is scored exactly once, after the model is finalised, for the metrics reported in this document.

10. Class Imbalance Handling

Positive rate: 8.11% overall, 6.60% in train, 6.02% in test. Two treatments were trained and compared for every model family rather than assuming one is superior:

- **Class weighting** — `class_weight='balanced'` for Logistic Regression / Random Forest, `scale_pos_weight=14.16` for XGBoost (computed as `negatives / positives` on the training set);
- **Threshold tuning only** — no re-weighting; the decision threshold is swept on validation and chosen to maximise F1 (and separately, expected cost).

SMOTE deliberately not used. Synthesising minority-class rows by interpolating between existing orders would mix information across time periods in a temporally ordered dataset — a synthetic order between a 2017 order and a 2018 order has no honest meaning. The proposal explicitly calls out class weighting and threshold tuning; both were implemented and compared instead.

11. Model Comparison & Selected Model

All six configurations, validation and test, reported without exception:

Model	Split	ROC-AUC	PR-AUC	Precision	Recall	F1	Brier	Threshold
logistic_regression_weighted	validation	0.7009	0.2836	0.2522	0.5778	0.3512	0.1361	0.32
logistic_regression_weighted	test	0.7552	0.1491	0.1568	0.4778	0.2362	0.0811	0.32
random_forest_weighted	validation	0.7068	0.2991	0.3002	0.4707	0.3666	0.1489	0.39
random_forest_weighted	test	0.6803	0.1140	0.0977	0.5780	0.1672	0.1503	0.39
xgboost_weighted	validation	0.7107	0.2886	0.2753	0.5472	0.3663	0.1712	0.47
xgboost_weighted	test	0.6808	0.1120	0.0988	0.6059	0.1699	0.1924	0.47
logistic_regression_threshold_only	validation	0.7190	0.3105	0.4305	0.1935	0.2670	0.1368	0.05
logistic_regression_threshold_only	test	0.7541	0.1442	0.1799	0.1436	0.1597	0.0572	0.05
random_forest_threshold_only (selected)	validation	0.6987	0.3138	0.2953	0.4731	0.3636	0.1202	0.13
random_forest_threshold_only (selected)	test	0.7261	0.1347	0.1061	0.7474	0.1859	0.0591	0.13
xgboost_threshold_only	validation	0.7108	0.3076	0.2875	0.5419	0.3757	0.1193	0.11
xgboost_threshold_only	test	0.6963	0.1216	0.0980	0.6705	0.1710	0.0587	0.11

Selection rule: the model with the highest **validation** PR-AUC is chosen — PR-AUC rather than accuracy or ROC-AUC alone, because with an 8% positive rate, accuracy is dominated by the majority class and PR-AUC better reflects ranking quality on the minority (late) class. Selection uses validation only; the test column is reported afterwards and never used to choose the model.

Selected: Random Forest, no class weighting (validation PR-AUC 0.3138). Notably, the Logistic Regression baseline generalised *better* to the test window (ROC-AUC 0.7552 vs 0.7261) even though it lost on the validation selection criterion. Both results are reported rather than the weaker one being omitted.

12. Evaluation Metrics (Measured Results)

12.1 Test-set headline (threshold 0.13, tuned on validation for F1)

ROC-AUC 0.7261	PR-AUC 0.1347 2.2× the 6.02% base rate	RECALL 74.74%	PRECISION 10.61%	BRIER SCORE 0.0591 lower = better calibrated
--------------------------	--	-------------------------	----------------------------	--

12.2 Confusion matrix (test, threshold 0.13)

	Predicted on-time	Predicted late
Actually on-time	18,029 (TN)	12,185 (FP)
Actually late	489 (FN)	1,447 (TP)

12.3 Cost-optimal threshold (0.09) — for comparison

Tuned on validation to minimise expected business cost rather than F1: recall 97.57%, precision 6.80%. This threshold does **not** transfer well to test (very high false-alert volume, TN 4,318 / FP 25,896) — a direct, honest illustration of the regime shift between validation and test discussed in Section 9.

12.4 Honest interpretation

ROC-AUC 0.726 and a PR-AUC 2.2× the base rate indicate the model **ranks** delay risk usefully — it separates late from on-time orders far better than chance. However, precision of ~11% at the working threshold means a flagged order is still more often on-time than late. **Accuracy is not used as a selection metric:** a trivial "always predict on-time" classifier would score ~94% accuracy on this data and be operationally useless. DelayShield is positioned as a **triage and prioritisation** tool, not a certainty oracle — its value comes from the risk-band lift (Section 13) and the policy simulation (Section 17), not from raw precision.

13. Risk Classification

Fixed thresholds of 0.30 / 0.60 (a naive first choice) would leave the HIGH band empty: the selected model's test-set probabilities span only **0.0397 to 0.2721** (median 0.1230). Risk bands are therefore derived from the **validation-set probability distribution** instead of arbitrary fixed cut-offs:

Band	Rule	Cut-off used
LOW	below the 75th percentile of validation probabilities	$p < 0.1273$
MEDIUM	75th–92nd percentile	$0.1273 \leq p < 0.1671$
HIGH	top ~8% of predicted risk	$p \geq 0.1671$

These are **operational capacity choices**, not statistically validated boundaries, and both cut-offs are live sliders in the dashboard. Measured on the test window, the observed delay rate rises monotonically across the three bands — the core sanity check that the model carries real signal:

Band	Orders	Observed delay rate	Lift vs population (6.02%)
LOW	17,587	2.47%	0.41×
MEDIUM	10,292	8.54%	1.42×
HIGH	4,271	14.56%	2.42×

14. Explainable AI (SHAP)

SHAP (SHapley Additive exPlanations) is applied to the selected Random Forest model via `TreeExplainer`, which is exact (not approximated) for tree ensembles and fast enough for live use in the dashboard.

14.1 Local explanations

For every single-order prediction, the dashboard shows signed, additive per-feature contributions: positive SHAP values push the prediction toward "late", negative values pull it toward "on-time". The contributions sum with the base rate to reproduce the model's exact output probability, so an operator can read off *why* a specific order is flagged — not just that it is.

14.2 Global explanations

Mean absolute SHAP value across a sample of 400 scored orders identifies the features the model relies on most overall. The top-ranked features are typically platform-wide workload, destination-state historical delay rate, and the engineered interaction features (`lead_time_ratio`, `lead_time_slack_days`, `distance_per_promised_day`) — confirming the model is not relying on a single spurious column.

14.3 Reviewer question: why SHAP?

The proposal calls for explainability at the individual-prediction level, not just a global ranking. SHAP is additive and locally accurate by construction, which lets an operator trust the stated reason for a specific alert rather than a model-wide generalisation that may not hold for that order.

15. Supplier Prioritization Engine

Every seller (2,959 total) is scored on five components, each min-max normalised to 0–1 (1 = best) before combination:

```
Supplier Score = 0.35 × reliability      (historical on-time rate)
                + 0.25 × predicted delay perf. (inverted mean model risk)
                + 0.15 × delivery speed      (inverted avg delivery days)
                + 0.10 × freight performance (inverted freight / order-value ratio)
                + 0.15 × customer feedback   (mean review score)
```

Scaled to 0–100. Tiers: **WATCHLIST** below 40, **STANDARD** 40–60, **PREFERRED** above 60. Sellers with fewer than 10 delivered orders are still ranked but flagged `low_volume_flag`, since their historical rates are statistically noisy at that sample size.

Initial operational weighting, not a validated formula. All five weights are exposed as live sliders on the Supplier Ranking dashboard page; the ranking recomputes instantly when a reviewer changes them, which demonstrates the formula is genuinely parameterised rather than hard-coded.

Customer review scores are used **only** in this retrospective scorecard — never as an input to the forward-looking delay-prediction model, consistent with the leakage policy in Section 5.

16. Action Recommendation Engine

Rule-based by design, so the mapping from risk to action is auditable by operations staff rather than opaque:

Risk	Headline	Actions
LOW	Normal processing	Standard flow; no customer pre-notification; standard tracking cadence.
MEDIUM	Monitor and buffer	Increased shipment tracking; +2-day schedule buffer; weekly supplier performance monitoring.
HIGH	Proactive intervention	Proactive customer communication; alternate-supplier review; +5-day buffer with carrier re-plan; daily tracking escalation.

Context rules extend the base playbook: a supplier delay rate above 15% adds a corrective-action review; WATCHLIST-tier suppliers trigger a restriction on new high-value orders; order value above 1,000 adds priority-queue handling; predicted probability ≥ 0.8 suggests a split shipment or carrier upgrade.

17. Business Cost Model & Policy Simulation

17.1 Assumptions (explicitly editable, never presented as observed Olist costs)

Assumption	Default value
Cost of a late delivery	1,200
Cost of a false alert	30
Cost of an intervention	50
Proactive intervention effectiveness	60% of damage avoided
Reactive intervention effectiveness	15% of damage avoided

17.2 The two policies compared

Reactive — nothing happens at confirmation; every order that ends up late gets a clean-up intervention only after the delay is visible, recovering just the reactive-effectiveness share of the damage.

DelayShield (risk-aware) — every order with predicted probability at or above the alert threshold gets a proactive intervention at confirmation time, recovering the higher proactive-effectiveness share. Wrongly flagged orders cost a false alert plus the intervention. Orders that slip through undetected still receive the reactive treatment afterwards, keeping the comparison fair — both policies run on the identical set of held-out orders.

17.3 Break-even precision (closed form)

```
break-even precision = (false_alert_cost + intervention_cost)
                      / ((proactive_eff - reactive_eff) × late_cost + false_alert_cost + intervention_cost)
                      = (30 + 50) / ((0.60 - 0.15) × 1200 + 30 + 50)
                      = 80 / 620 = 12.9%
```

Alert precision must exceed 12.9% for the risk-aware policy to be cost-positive under the default assumptions. This is not a hand-picked number — it falls directly out of the cost model.

17.4 Measured outcome on the test window

At threshold 0.18 (precision 16.4%, above break-even): 435 of 1,936 late orders caught early (22.5% recall), saving **≈57.8k** currency units versus reactive. At threshold 0.10 (precision 7.4%, below break-even): the policy **loses ≈839k** versus reactive — too many false alerts for the mitigation gain to offset. The dashboard's threshold-sweep chart displays this entire curve, including the loss region, rather than only the favourable point.

The validation-optimal cost threshold (0.09) does not transfer to the test window — a concrete, quantified demonstration of the regime shift discussed in Section 9, shown honestly rather than smoothed over.

18. Streamlit Dashboard

Seven pages, styled as an enterprise supply-chain risk platform (KPI cards, sidebar navigation, consistent risk-colour coding, live-editable parameters):

Page	Contents
Dashboard	KPI row, monthly actual-vs-predicted risk trend, risk mix, risk-band lift sanity table.
Order Risk Prediction	Live scoring form → probability, risk band, gauge, SHAP explanation, recommended action, supplier context.
Supplier Ranking	Live weight sliders, volume/state filters, ranked scorecard, volume-vs-delay scatter, worst-supplier chart.
Risk Alerts	Filterable high-risk order queue, alert-quality lift vs population, CSV export.
Analytics	Late vs on-time split, probability distribution, delay by category/state/seller/month, global SHAP importance.
Policy Simulation	Editable cost assumptions, reactive vs DelayShield KPI comparison, cost breakdown, threshold sweep, sensitivity analysis.
Model & Method	Chronological split table, all six model results, calibration curve, confusion matrix, leakage policy statement, stated limitations.

Sidebar controls apply globally across every page: data scope (test / validation / full history) and the two risk-band thresholds, so the reviewer can move a slider once and see every page respond consistently.

Launch

```
pip install -r requirements.txt
python src/train.py           # trains model, writes all artefacts (~50s)
streamlit run dashboard/app.py # launches on http://localhost:8501
```

19. Project Structure

```
DelayShield/
├─ data/
│  ├─ raw/          9 original Olist CSVs
│  ├─ processed/    model_dataset.csv, scored_orders.csv, supplier_metrics.csv
│  └─ sample/       small samples for inspection
├─ models/          delay_model.pkl, feature_columns.pkl, model_metadata.json
├─ notebooks/       01_eda.ipynb, 02_model_development.ipynb
├─ reports/         eda_report.md, model_comparison.csv
├─ src/
│  ├─ data_loader.py  raw table loading
│  ├─ preprocessing.py clean, integrate, target
│  ├─ feature_engineering.py time-aware leakage-free features
│  ├─ train.py        split, train, evaluate, persist
│  ├─ predict.py      single/batch prediction engine
│  ├─ explain.py      SHAP explanations
│  ├─ supplier_scoring.py scorecard + weighted score
│  ├─ recommendations.py action playbook
│  ├─ simulation.py   policy comparison
│  └─ eda_report.py   dataset inspection report generator
├─ dashboard/app.py  7-page Streamlit application
├─ requirements.txt
├─ run.bat / run.sh
└─ README.md
```

20. Limitations

- **Ranking, not certainty.** Test ROC-AUC ≈ 0.73 means the model orders risk usefully, but individual predictions carry wide uncertainty; alert precision at the working threshold ($\approx 11\%$) is far below 50%, so alerts function as a triage queue rather than a verdict.
- **Regime shift.** The 2018 Brazilian carrier disruption changes the delay base rate between validation and test windows; thresholds tuned on one window do not transfer perfectly, as quantified in Sections 9, 12 and 17.
- **Costs are assumptions.** Olist publishes no cost or intervention data; every currency figure in this report and the dashboard is configurable and explicitly labelled ASSUMPTION.
- **Intervention effectiveness is assumed, not measured.** Proving the real effect of a proactive intervention requires a controlled A/B experiment on flagged orders, which historical data alone cannot supply.
- **Single-snapshot history.** The live prediction form uses each seller's latest historical statistics at the time the model was trained; a production deployment would need a continuously updated feature store.
- **Geography and period.** Findings are specific to the Brazilian market, 2016–2018; they do not transfer unchanged to another market, carrier network or time period.

21. Reviewer Q&A

Why Olist?

Named explicitly in the approved proposal; one of the few large public e-commerce datasets carrying both promised and actual delivery dates alongside seller, product, freight, location and review data.

Why this target?

`actual > estimated` is the SLA breach the customer actually experiences. It needs no arbitrary threshold and both timestamps exist for every delivered order.

Why a time-based split?

Delay risk is non-stationary (train 6.60%, validation 14.58%, test 6.02% late). A random split would train on future orders and the same seller's later performance — impossible at real prediction time and would inflate every reported metric.

How was leakage avoided?

Two rules (Section 5): no post-outcome column is ever a feature; and every history feature uses only orders already *delivered* before the current purchase timestamp, enforced by construction in `_lagged_outcome_stats()`.

Why Random Forest / XGBoost alongside Logistic Regression?

The relationships are non-linear and interaction-heavy — e.g. promised lead time matters relative to the seller's usual speed and the route distance. Logistic Regression is the interpretable baseline; both tree families were trained, and all six configurations are reported rather than only the winner.

Why SHAP?

It gives per-order, signed, additive attributions — an operator needs "why is this specific order risky", not only a global feature ranking. Tree SHAP is exact for the selected model and fast enough for live dashboard use.

How is the supplier score calculated?

Five min-max-normalised components combined with configurable weights (Section 15), exposed as live sliders and explicitly labelled an initial operational weighting rather than a validated formula.

How are risk thresholds selected?

From validation-set probability quantiles (75th / 92nd), because a class-imbalanced model rarely emits probabilities above 0.3; fixed thresholds would leave the HIGH band empty. They are adjustable live in the dashboard, and the cost-optimal threshold is computed separately.

How does the simulation work?

Identical held-out orders scored under two policies (reactive vs risk-aware) with explicit, editable cost assumptions, plus an analytically derived break-even precision and a full threshold-sensitivity sweep (Section 17).

What are the limitations?

See Section 20 — ranking quality rather than certainty, an unmodelled regime shift, assumption-based costs, an unproven intervention effect, and a static history snapshot at prediction time.

What is the next production step?

(1) Serve the model behind an API with a continuously updated feature store recomputing lagged statistics in real time; (2) run a controlled A/B test on flagged orders to measure real intervention effectiveness instead of assuming it; (3) add drift and regime-change monitoring with scheduled retraining; (4) re-tune thresholds against measured, rather than assumed, intervention costs.

22. Conclusion & Future Work

DelayShield demonstrates a complete, working decision-support pipeline built from raw public data to an interactive application in a single development cycle: data cleaning and integration, leakage-free time-aware feature engineering, a disciplined chronological evaluation methodology, a documented model comparison, explainability, supplier prioritisation, a rule-based recommendation engine, and a transparent, assumption-driven policy simulation — all exposed through a live Streamlit dashboard.

The measured results are reported without embellishment: a test ROC-AUC of 0.726 and a risk-band lift of 2.42× on the highest-risk decile demonstrate genuine, useful ranking ability, while the low absolute alert precision ($\approx 11\%$) and the documented regime shift between validation and test windows are stated as honest limitations rather than smoothed over. The policy simulation shows, with an analytically derived break-even point, exactly the operating region in which a risk-aware intervention policy is cost-positive versus a purely reactive one.

Immediate next steps for a production deployment are a real-time feature store, a controlled experiment to measure actual intervention effectiveness, and continuous drift monitoring — all identified explicitly in Section 20 rather than left implicit.